

Torneo Universitario de Videojuegos

Ejercicio #2 Arreglos Básicos

Análisis

1. ¿Qué propiedades y métodos tendrá cada clase?

a. Jugador

- Representa al jugador del torneo y contiene toda la información relacionada con sus partidas.

Atributo	Tipo	Visibilidad	Propósito
nombre	String	private	Almacena el nombre del jugador
nickname	String	private	Almacena el nombre de usuario
edad	int	private	Almacena la edad
puntajes	int[]	private	Almacena los puntajes de las partidas
partidasJugadas	int	private	Indica cuántas partidas han sido registradas

b. Driver Program

- Contiene el Main() y se encarga de interactuar con el usuario mediante el menú

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

a. Clase Jugador

Método	Retorno	Parámetros	Propósito
Jugador()	— ninguno	Constructor	
Jugador(String, String, int)	— nombre, nickname, edad	Inicializar jugador	
registrarPartida(int)	boolean puntaje	Registrar un nuevo puntaje	
consultarPuntajes()	void ninguno	Mostrar partidas registradas	
consultarPartida(int)	int número de partida	Obtener un puntaje específico	
modificarPuntaje(int, int)	boolean partida, nuevo puntaje	Modificar un puntaje	
calcularPromedio()	double ninguno	Calcular promedio	
obtenerMayor()	int ninguno	Obtener mayor puntaje	

<code>obtenerMenor()</code>	<code>int</code>	<code>ninguno</code>	Obtener menor puntaje
<code>getPartidasJugadas()</code>	<code>int</code>	<code>ninguno</code>	Obtener partidas jugadas
<code>getPartidasDisponibles()</code>	<code>int</code>	<code>ninguno</code>	Obtener espacios disponibles

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo? ¿Qué tipo de datos almacenará?
 - a. PartidasJugadas = Cantidad de partidas jugadas = int
 - b. Puntajes = Puntaje obtenido en cada partida = int
4. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
 - a. Los atributos de la clase Jugador deben utilizar el modificador private, con el objetivo de proteger la información del objeto y mantener el encapsulamiento.
 - b. Los métodos que necesiten ser utilizados desde el Main deberán ser public. De esta manera, el Main podrá solicitar las operaciones de la clase Jugador sin acceder directamente a sus atributos.
 - c. El método main() de la clase main deberá ser public static void, debido a que es el punto de inicio de la ejecución del programa.
5. ¿Qué parámetros serán requeridos por los métodos en sus clases?
 - a. El constructor de Jugador recibirá como parámetros el nombre, nickname y edad del jugador.
 - b. El método para registrar una partida recibirá como parámetro el puntaje obtenido.
 - c. El método para consultar una partida recibirá el número de la partida que se desea consultar.
 - d. El método para modificar un puntaje recibirá dos parámetros: el número de la partida que se desea modificar y el nuevo puntaje.
 - e. Los métodos para consultar todos los puntajes, calcular el promedio, obtener el mayor y menor puntaje y consultar las partidas disponibles no necesitarán parámetros adicionales, ya que utilizarán la información almacenada dentro del objeto
6. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
 - a. Los valores iniciales del objeto Jugador serán proporcionados mediante un constructor que recibirá el nombre, nickname y edad ingresados por el usuario.
 - b. Al crear un nuevo jugador, el arreglo de puntajes tendrá una capacidad de 10 posiciones y partidasJugadas iniciará en 0, debido a que el nuevo jugador comienza sin partidas registradas.
 - c.
7. ¿Cómo determinará cuál es la siguiente posición disponible dentro del arreglo?
 - a. La siguiente posición disponible se determinará utilizando el atributo partidasJugadas.
 - b. Como las posiciones del arreglo comienzan en el índice 0, si el jugador tiene 0 partidas, el siguiente puntaje se almacenará en la posición 0. Si tiene 1 partida, se almacenará en la posición 1, y así sucesivamente.

8. ¿Cómo recorrerá únicamente las posiciones del arreglo que contienen puntajes registrados?
- a. Para recorrer únicamente las posiciones utilizadas se utilizará la cantidad de partidas jugadas como límite del ciclo.
 - b. Por ejemplo, se utilizará un ciclo que comience en el índice 0 y continúe mientras el índice sea menor que partidasJugadas.
 - c. De esta manera, si el jugador solamente ha registrado tres partidas, únicamente se recorrerán las posiciones 0, 1 y 2, sin mostrar las posiciones restantes del arreglo como partidas jugadas.

Diseño

UML

Clase Jugador

Visibilidad	Atributo	Tipo
-	nombre	String
-	nickname	String
-	edad	int
-	puntajes	int[]
-	partidasJugadas	int

Métodos

Visibilidad	Método	Retorno
+	Jugador()	—
+	Jugador(String, String, int)	—
+	registrarPartida(int)	boolean
+	consultarPuntajes()	void
+	consultarPartida(int)	int
+	modificarPuntaje(int, int)	boolean
+	calcularPromedio()	double
+	obtenerMayor()	int
+	obtenerMenor()	int
+	getPartidasJugadas()	int
+	getPartidasDisponibles()	int

Clase Main

Visibilidad	Método	Retorno
+	main(String[])	void

	Jugador	
	- nombre: String	
	- nickname: String	
	- edad: int	
	- puntajes: int[]	
	- partidasJugadas: int	
	+ Jugador()	
	+ Jugador(nombre: String,	
	nickname: String, edad: int)	
	+ registrarPartida(puntaje: int): boolean	
	+ consultarPuntajes(): void	
	+ consultarPartida(partida: int): int	
	+ modificarPuntaje(partida: int,	
	nuevoPuntaje: int):	
	boolean	
	+ calcularPromedio(): double	
	+ obtenerMayor(): int	
	+ obtenerMenor(): int	
	+ getPartidasJugadas(): int	
	+ getPartidasDisponibles(): int	

